Sémaphore (informatique)

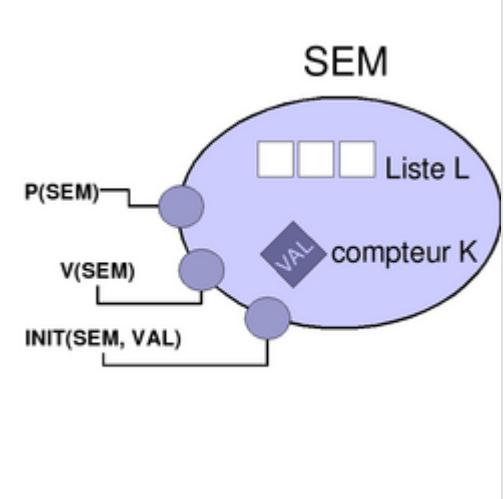
Un **sémaphore** est une variable (ou un type de donnée abstrait) partagée par différents « acteurs », qui garantit que ceux-ci ne peuvent y accéder que de façon séquentielle à travers des opérations atomiques, et constitue la méthode utilisée couramment pour restreindre l'accès à des ressources partagées (par exemple un espace de stockage) et synchroniser les processus dans un environnement de programmation concurrente. Le sémaphore a été inventé par Edsger Dijkstra[1] et utilisé pour la première fois dans le système d'exploitation THE Operating system.

Les sémaphores fournissent la solution la plus courante pour le problème du « dîner des philosophes », bien qu'ils ne permettent pas d'éviter tous les interblocages (ou *deadlocks*). Pour pouvoir exister sous forme logicielle, ils nécessitent une implémentation matérielle (au niveau du microprocesseur), permettant de *tester et modifier* la variable protégée au cours d'un cycle insécable. En effet, dans un contexte de multiprogrammation, on ne peut prendre le risque de voir la variable modifiée par un autre processus juste après que le processus courant vient de la tester et avant qu'il ne la modifie.

Opérations

Les trois opérations prises en charge sont `Init`, `P` et `V`.

`P` et `V` du néerlandais *Proberen* et *Verhogen* signifient « sonder » et « incrémenter, augmenter ». La valeur initiale d'un sémaphore est le nombre d'unités de ressource[2] (exemple : mémoires, imprimantes…) ; elle est décrémentée à chaque fois qu'un processus exécute l'opération P. Si elle est positive, elle représente donc le nombre de ressources libres, sinon si elle est négative sa valeur absolue correspond au nombre de processus en attente.



Le sémaphore SEM, sa liste L et son compteur K, SEM est accessible aux opérations : `* Init(sem, val)` ; `* P(sem)` ; `* V(sem)`.

- L'opération `P` met en attente le processus courant jusqu'à ce qu'une ressource soit disponible, ressource qui sera immédiatement allouée au processus courant.
- `V` est l'opération inverse; elle rend simplement une ressource disponible à nouveau après que le processus a terminé de l'utiliser.
- `Init` est seulement utilisé pour initialiser le sémaphore. Cette opération ne doit être utilisée qu'une seule et unique fois.

Les opérations `P` et `V` doivent être atomiques, ce qui signifie qu'elles doivent s'exécuter en une fois et ne peuvent être interrompues entre deux « sous opérations ». On ne peut pas avoir une situation dans laquelle deux processus testeraient (quasi) simultanément la même ressource et puis la réserveraient

(quasi) simultanément. Le premier processus doit tester et réserver la ressource (ou être mis en attente) dans une seule et même opération, et ensuite seulement, le second peut tester et réserver (ou être mis en attente).

Dans les livres en anglais, les opérations `V` et `P` sont quelquefois appelées respectivement *up* et *down*. En conception logicielle, elles sont appelées *signal* et *wait* ou *release* et *take*. De façon informelle, un bon moyen de se rappeler laquelle fait quoi est le procédé mnémotechnique les associant à *Puis-je ?* ou *Prendre* et *Vas-y !* ou *Vendre*.

Pour éviter l'attente, un sémaphore peut avoir une file de processus associée (généralement une file du type FIFO). Si un processus exécute l'opération `P` sur un sémaphore qui a la valeur zéro, le processus est ajouté à la file du sémaphore. Quand un autre processus incrémente le sémaphore en exécutant l'opération `V`, et qu'il y a des processus dans la file, l'un d'eux est retiré de la file et reprend la suite de son exécution.

`Init(sem, val)`

Initialisation d'un sémaphore :

```
function Init(semaphore sem, int val){
   disable_interrupt;
   sem.K = val;
   enable_interrupt;
}
```

La norme POSIX définit la fonction `sem_init` pour cela[3].

`P(sem)`

```
function P(semaphore sem){
   disable_interrupt;
   if (sem.K <= 0){
       L.suivant = processus_courant;
       processus_courant.état = bloque;
       reordonnancement = vrai;
   }
   sem.K = sem.K - 1;
   enable_interrupt;
}
```

La norme POSIX définit la fonction `sem_wait` pour cela[4].

`V(sem)`

```
function V(semaphore sem){
   disable_interrupt;
   sem.K = sem.K+1;
   if (not L.vide){
       processus_réveillé = L.tête;
       processus_réveillé.état = prêt;
       reordonnancement = vrai;
   }
   enable_interrupt;
}
```

La norme POSIX définit la fonction `sem_post` pour cela.

L'incrémentation de la variable *K* ne doit pas être interrompue, et l'opération P ne doit pas être interrompue quand *K* est différent de 0. Ceci peut être fait par une instruction spéciale ou en ignorant les interruptions afin d'empêcher d'autres processus de devenir actifs. Les sémaphores peuvent être utilisés pour la synchronisation des entrées/sorties.

Utilisation

Les sémaphores sont toujours utilisés dans les langages de programmation qui n'implémentent pas intrinsèquement d'autres formes de synchronisation. Ils sont le mécanisme primitif de synchronisation de beaucoup de systèmes d'exploitation. La tendance dans le développement des langages de programmation est de s'orienter vers des formes plus structurées de synchronisation comme les moniteurs. Outre les problèmes d'interblocages qu'ils peuvent provoquer, les sémaphores ne protègent pas les programmeurs

de l'erreur courante qui consiste à bloquer par un processus un sémaphore qui est déjà bloqué par ce même processus, et d'oublier de libérer un sémaphore qui a été bloqué. Hoare, Hansen, Andrews, Wirth, et même Dijkstra ont jugé le sémaphore obsolète[réf. nécessaire].

Exemples

Sémaphores bloquants

Outre les sémaphores à compteur interne, il existe également les sémaphores bloquants. Un sémaphore bloquant est un sémaphore qui est initialisé avec la valeur 0. Ceci a pour effet de bloquer n'importe quel *thread* qui effectue P(S) tant qu'un autre *thread* n'aura pas fait un V(S). Ce type d'utilisation est très utile lorsque l'on a besoin de contrôler l'ordre d'exécution entre *threads*. Cette utilisation des sémaphores permet de réaliser des barrières de synchronisation.

Exclusion mutuelle

Il existe également le sémaphore binaire qui est une exclusion mutuelle (alias mutex). Il est toujours initialisé avec la valeur 1.

Lecteurs-rédacteurs

Un problème classique pouvant être résolu à l'aide des sémaphores est le problème des lecteurs/rédacteurs. Ce problème traite de l'accès concurrent en lecture et en écriture à une ressource. Plusieurs processus légers (*thread*) peuvent lire en même temps la ressource, mais il ne peut y avoir qu'un et un seul thread en écriture.

Producteurs-consommateurs

Lorsque des processus légers souhaitent communiquer entre eux, ils peuvent le faire par l'intermédiaire d'une file. Il faut définir le comportement à avoir lorsqu'un thread souhaite lire depuis la file lorsque celle-ci est vide et lorsqu'un thread souhaite écrire dans la file mais que celle-ci est pleine.

Autres problèmes

Les sémaphores servant notamment à effectuer de la synchronisation, ils peuvent conduire à des situations indésirables, par exemple :

- Interblocage ;
- Inversion de priorité.

Notes et références

1. (en) Genuys, F.,, NATO Science Committee. et Université de Grenoble., *Programming languages;*, Academic P, 1968 (ISBN 0-12-279750-7 et 978-0-12-279750-7, OCLC 1068 (https://worldcat.org/fr/title/1068), lire en ligne (https://research.tue.nl/en/publicat

ions/co-operating-sequential-processes)), p. 43-112 (Co-operating sequential processes by Dijkstra, E.W.)
2. *Technique Et Science Informatiques : TSI., Volume 5*, Dunod (lire en ligne (https://books.google.fr/books?id=oCxMAAAAYAAJ&dq=%22valeur+initiale+du+s%C3%A9maphore%22)).
3. « SEM_INIT (http://manpagesfr.free.fr/man/man3/sem_init.3.html) », sur *manpagesfr.free.fr* (consulté le 13 avril 2023).
4. (http://albertllongueras.cat/albert_universitat/Telecos/Pc/solExamP09.pdf) **[PDF]** page 2.

Ce document provient de « https://fr.wikipedia.org/w/index.php?title=Sémaphore_(informatique)&oldid=237894951 ».